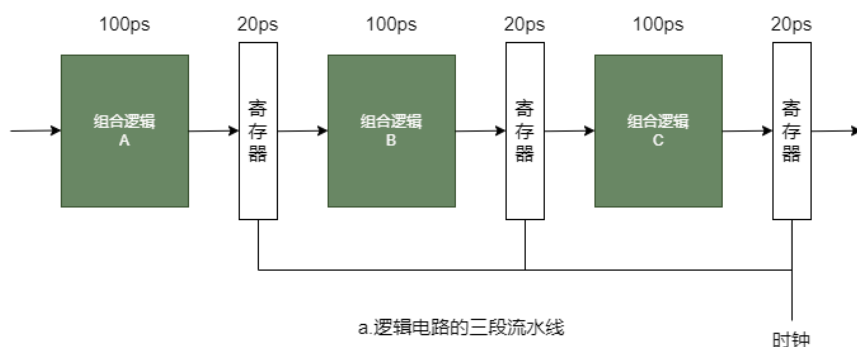


从电路底层来了解CPU中断机制



倚翹

想当科学家

📖 收录于 · 模拟IC设计实践 >

创作声明：包含 AI 辅助创作 ∨

273 人赞同了该文章 >

中断 (interrupt)，在计算机工作的时候往往都需要中断处理。而为什么需要中断呢？试想一下，现在的计算机一般都具有多任务处理能力，我们日常用的电脑一般都同一时刻进行着几十上百个进程。想想一下如果现在我们需要进行键盘打字和屏幕输出文字的这种情况，当键盘输入一个文字后，指令进入CPU处理，再到输出到屏幕。可能我们觉得很快，但是如果如果没有中断处理，一个CPU核只处理一个命令，那我们就会等到前一个文字输出到屏幕上，我们才能进行下一个输入，这个速度是很慢的，所以**中断机制**⁺的出现使得我们更好地提升CPU core的性能。

而在讲解中断技术之前，我们应该了解如下几个概念，对于中断了解可能更加全面和升入。

时钟周期⁺、组合电路⁺与时序电路⁺

虽然，每个数字电路系统可能包含有组合电路，但是在实际应用中绝大多数的系统还包括存储元件，而这样的电路被称之为时序电路。

而在学习数电的时候，我们了解过时序电路是由最基本的**逻辑门**⁺电路加上反馈逻辑回路或者器件而形成的电路（从输出到输入），与组合逻辑最大的区别在于时序电路有记忆功能。

而时序电路的输出与输入，并不只依赖于输入，有时候还包括上一个周期的电路状态，因此映射到电路上就包含大量的存储单元。而时序电路的特点之一还有，在时序电路中往往各个器件需要同时可进行同步动作。**触发器**⁺和**锁存器**⁺是响应时钟周期和调整整个电路工作动作一致的关键器件。在每一个存储单元电路上引入一个时钟脉冲信号（CLK）作为控制信号，只有当时钟信号上升沿到来的时候，电路才被触发，并根据输入信号的改变而改变自己的输出状态。

时钟周期

而言，一个节拍的时间，就被称之为时钟周期。

时钟周期是由CPU时钟定义的定长时间间隔，是CPU工作的最小时间单位。

由于时钟周期的出现，时序电路又被分为：

- 同步时序电路：存储电路内所有触发器的时钟接入都要接于同一个时钟脉冲源，所有触发器的状态变化都与所加时钟脉冲信号同步
- 异步时序电路：没有同一的时钟脉冲

而根据输出信号的特点又可以电路分为：

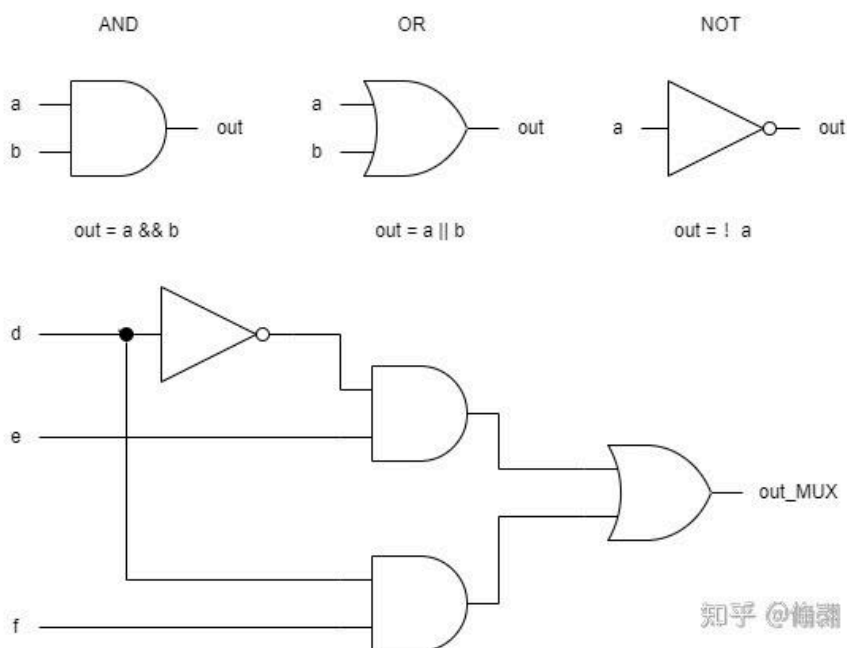
- Mealy型：输出信号不仅取决于存储电路装，而且还取决于输入变量
- Moore型：输出信号仅取决于存储电路的状态。

组合电路

在上面简述的这些例子中，可能会存在疑问，在接下来对上诉可能产生的疑问做解答：

什么是触发器和锁存器，他们是如何工作的？

而要回答这个疑问我们需要了解逻辑电路，逻辑电路最基础的基本单位就是：逻辑门。



逻辑门包含三种，与、或、非门。

逻辑门总是活动的，一旦一个门的输入变化了，在很短一个时间内，输出就会发送变化。使用很多逻辑门构建一张网，得到了一个实现复杂功能的逻辑计算块（computational Block），就称之为组合电路。而在设计组合电路必须遵守之下规则，在芯片设计中也非常需要注意：

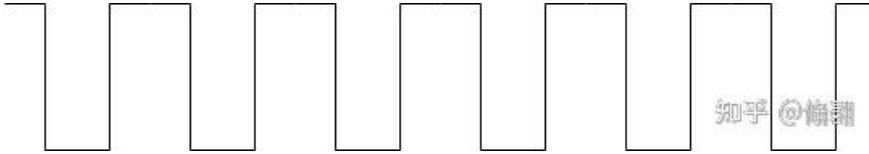
1. 每个逻辑门的输入必须接入一个系统的输入、每个储存单元的输出或者某个逻辑门的输出，在三者中选择一个。
2. 两个或者多个逻辑门的输出是不能够接在一起的，否则可能会产生互相矛盾的信号，造成错误或形成电路故障。
3. 这个网必须是无环的，不能形成回路，否则计算逻辑有歧义。

而逻辑门构成了组合电路，下面介绍电路中的控制信号

控制信号与时序电路

在上图的多路复用器中，有两个与门，上面的逻辑是 $d \& e$ ，下面的逻辑是 $d \& f$ 。也就是说：

- 当 $d=1$ 时， e 的输入是无法到达或门的，此时 f 为整个电路的输出
- 当 $d=0$ 时， f 的输入是无法到达或门的，此时 e 为整个电路的输出



所以 d 的输入电位决定着， e 、 f 哪个输入有效，通过上述例子再看控制信号是如何工作的。假想一下，简单的寄存器是如何依赖一个时钟控制信号工作的。

当一个时钟信号来临，电平从高到低再变高，以该信号为控制信号的寄存器内部的每一位输入都与该信号进行逻辑与后再进入寄存器，那么可以知道，在该电平为低的时候，任何信号都不能写入寄存器，等待时钟信号为高的时候才能写入。这就是一个简单的触发器，随着时钟信号改变状态，当然真实的触发器设计比这个复杂很多，这里不做展开交流。有了寄存器的存在，这种电路一般被称之为时序电路。

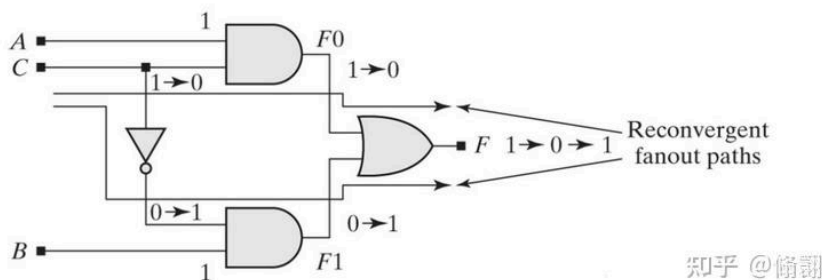
组合电路和时序电路的区别

1. 对毛刺的容忍

组合逻辑从基本上来讲，不储存任何信息，只是对简单的响应输入信号，哪怕这个组合逻辑很复杂。

时序逻辑是拥有自己的状态，时序电路每一个状态除了依赖输入外，还有可能依赖电路的上一个状态。而想要电路拥有自己的状态并可以基于这个状态进行计算，必须在组合逻辑电路中引入存储设备和控制存储设备的周期信号。

而引入存储设备容易理解，因为没有存储状态的载体，就不能呈现电路当时的状态。为什么要加入周期信号呢？下面举例来让大家理解：

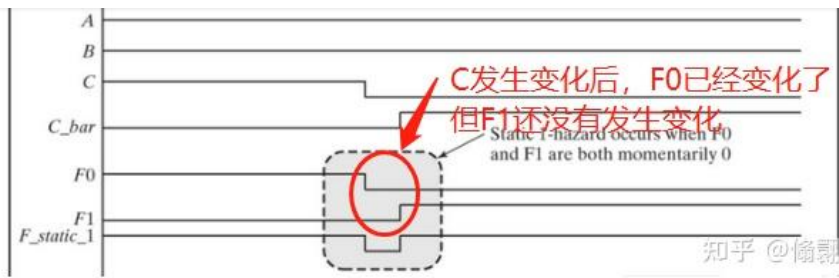


来看一个没有时钟信号的组合逻辑电路：

前面说过，逻辑门总是活动的，一旦一个逻辑门的输入改变，则输出会在很短的时间内发生改变。

但是需要注意的是，这个“很短时间内”的描述。由于元器件的质量/种类不同、路线的长度不同等物理因素的限制，不同的输入到达输出的时间是不一样的。

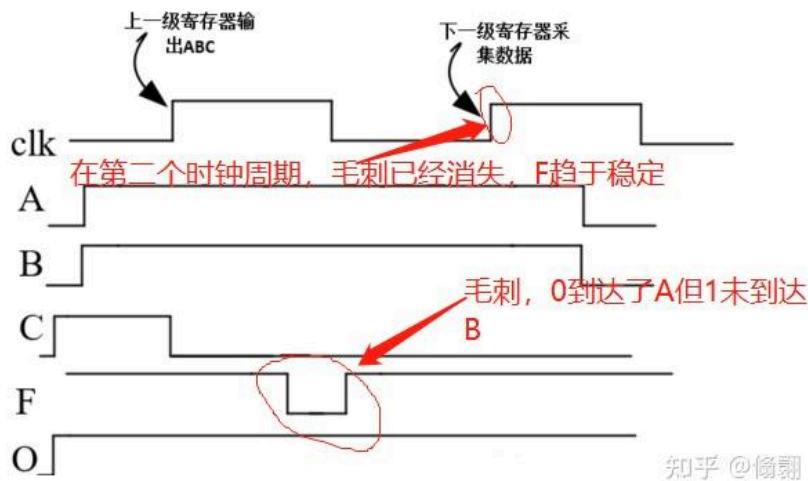
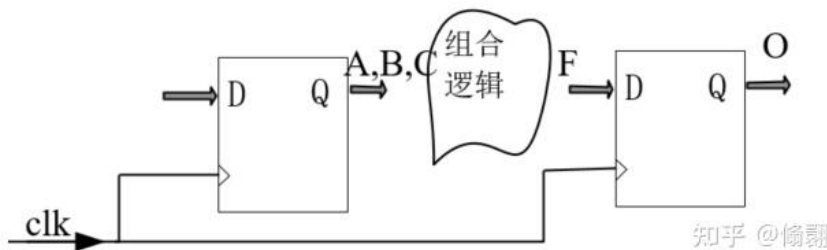
比如图中， c 输入到达下方的与门与到达上方的与门的两条路线中，到达下方与门的路线多出了一个非门。那么 c 信号到达上方的与门自然要比到达下方的与门的速度快。



所以当 c 信号发生改变时, 有一段时间内, F 端的输出是错误的, 因为 A&&C 已经到达 F 端但 B&&!C 还没有到达, 也就是说 F1 是比 F0 到达的慢的, 存在延迟, 如下图所示:

这种情况被称为“毛刺”。

虽然毛刺出现的时间是很短暂的, 但是对于一个电路系统的输出来说却是致命的。如果在发生毛刺的时间内将错误的输出写入存储器, 接下来的逻辑会一错再错并让人摸不到头脑。



而时序电路则不会出现上述问题, 将A/B/C的输入到F的输出看作一个完整的动作, 在一个时钟周期内完成。那么, A/B/C的输入将在时钟沿触发, F也将时钟沿采集结果。而在采集结果时, F的输出已经跨越了毛刺处于稳定状态。当然, 这样时一个钟周期内高电平持续的时间必须足够使 F 输出达到稳定状态。

这样, 下一个动作 (发生在下一个时钟周期) 如果基于 F 输出, 将得到正确的结果。这是时序电路与普通逻辑电路的区别之一: 对毛刺的容忍。

而在代码设计中, 也举例说明:

```
int a=0;
```

```
int b=a;
```

在这个简单的例子中, 需要a=0, 执行后b=a才有意义。a=0没有执行完成或未执行时, b=a的执行完全没有意义。这个程序执行的先后顺序, 和电路按照动作的先后顺序都是非常重要的。

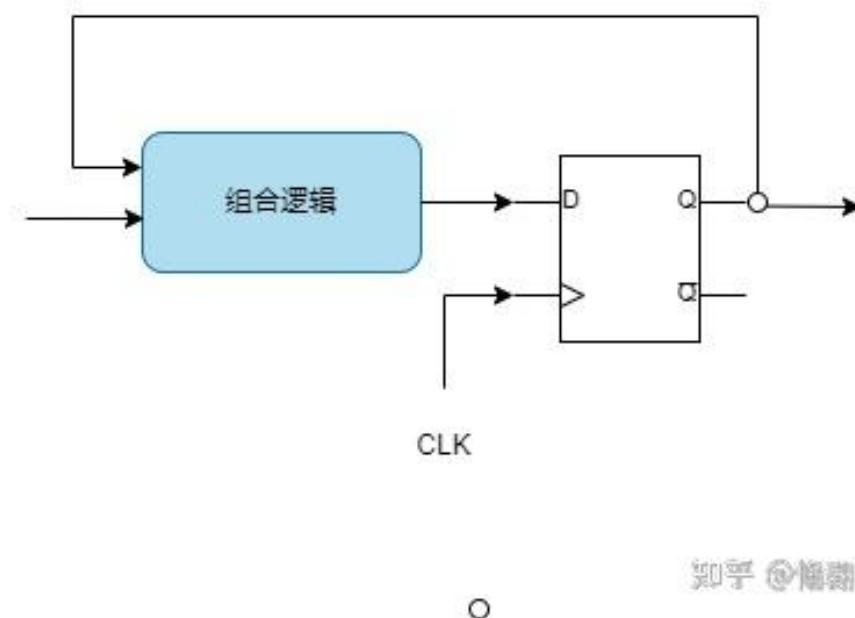
而时钟周期可以将一个个动作隔离开, 确保上一个动作已经执行完成后, 再执行下一个动作。这就是CPU所需要的, CPU执行一条条动作 (注意这里不是指的一条条指令, CPU的基本动作比指令更加细致, 尤其在引入流水线的操作后。基本动作指的是: 取指令、指令译码、执行指令、访存、写回、PC增加等), 周期就想一个个节拍, CPU跟随节拍快速而有条不紊的工作。因此一个时钟周期内必须满足CPU完成消耗最长时间的基本动作, 因此不同CPU需要的时钟频率也不一样, 确定一个CPU的时钟周期也是一个非常复杂的任务。

如果要实现一个计数器，如果用非时序电路实现是这样的：



这样的电路是无法使用的，电路的下一个输出依赖电路此时的状态，除了会产生上一节所说的毛刺之外，还会造成不可预期的后果，电路的逻辑存在死循环：

```
module count(
    input enable,
    output reg[7:0] out
);
always @(*) begin
    if(!enable)
        out = 8'h00;
    else
        out = out + 1'b1;
    end
endmodule
```



要支持反馈逻辑，必须使用寄存器将结果暂时存起来，由时钟沿控制数据镜像更新。

```
module count(
    input clk,
    input enable,
    output reg[7:0] out
```

```
always @(posedge clk) begin
```

```
if(!enable)
```

```
out = 8'h00;
```

```
else
```

```
out = out + 1'b1;
```

```
end
```

```
endmodule
```

在这里我们就清楚了时序电路的特性，下面介绍时序电路如何构成处理器。

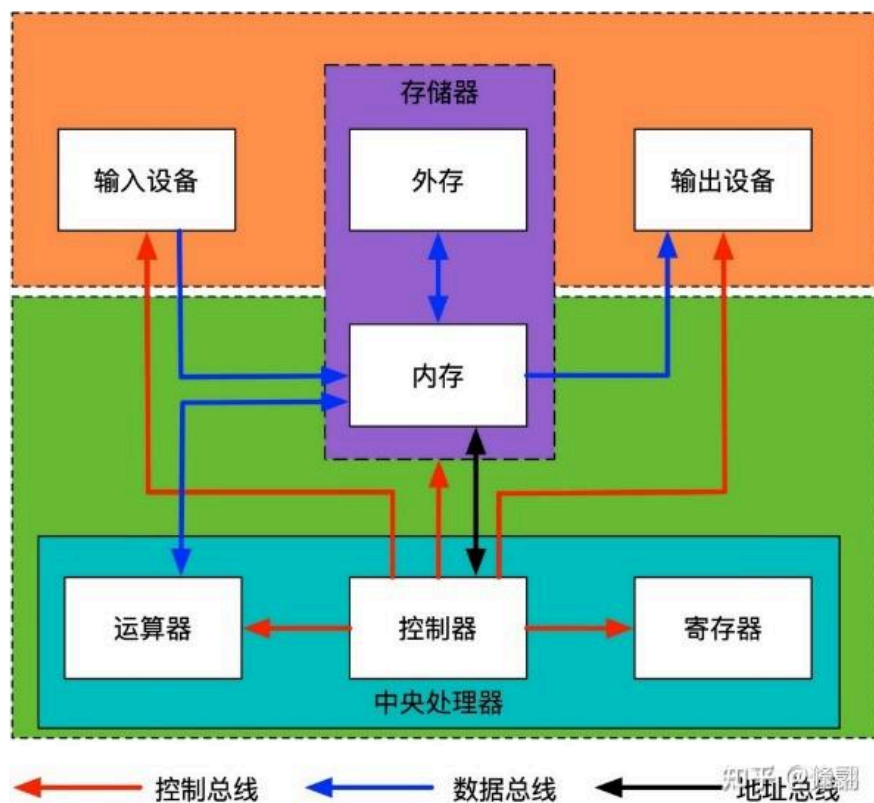
时序电路构成处理器

可以看到，一个最基本的处理器是这样一个电路：

1. 可以完成逻辑的运算。
2. 电路需要有自己的状态。
3. 每一个输出除了基于输入和处理逻辑外，还需要基于当前电路的状态。

而时序电路可以很好的符合这些特征，大多数时候，寄存器处于一种稳定状态，产生的输入等于它的当前状态。但当当前时钟脉冲处于低电位时，寄存器的输出仍保持不变。直到时钟脉冲变为高电位，输入信号便写入到寄存器中，成为下一个状态。直到下一个时钟上升沿，寄存器的状态和输出都不会发生改变。

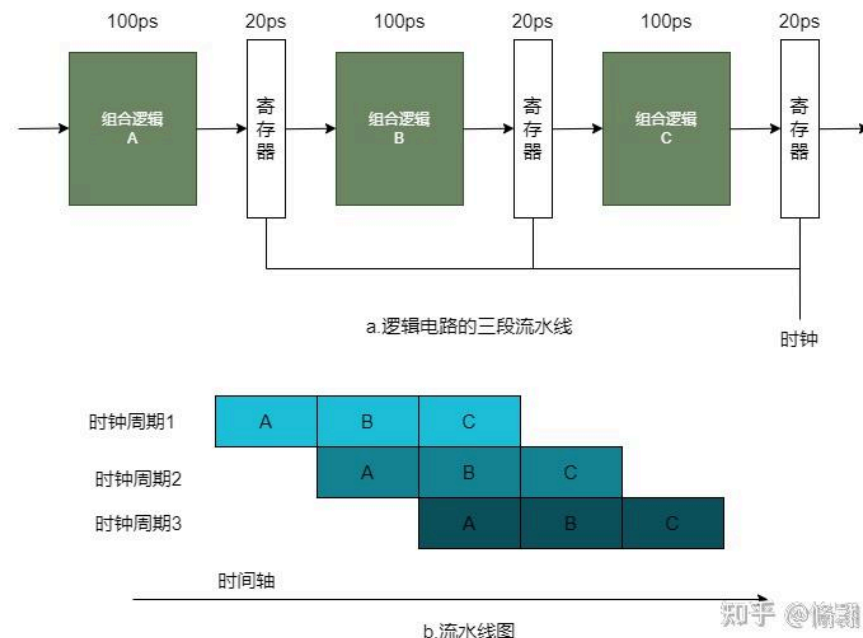
电信号畅通无阻的在组合电路中传播，而寄存器就成为这种传播的屏障。只有在每个时钟的上升沿时，信号才可以通过寄存器进入下一个组合电路。换个角度，时钟周期保证了每个周期结束时，这个周期内的输入已经完整的转化为了输出。而这个输出保存在寄存器内供下个周期的动作使用。时钟周期和寄存器的配合将电路要执行的动作与动作之间隔离开来。一个动作有条不紊的执行，周而复始。



流转。

流水线操作和周期

在节中不难看出，如果要按照上述的流程，计算机的一个周期时间太长而效率不高，因此，可以想象一下将负责各个动作的电路之间用寄存器隔离开来，一个时钟周期只执行一个简单的动作，而不是一个指令，这样可以大大增加电路整体的效率。



事实上，流水线操作便是这样做的，为了提升效率，流水线操作的层级可能很深，一个取指、译码、执行指令这个三个动作可能会被分成十几个以上的动作。这样一个周期内，就可以处理多条指令，而值得注意的是，这多条指令不是并发的，而是处于不同阶段的。下图我们给出简单的流水线操作便于大家理解。

寄存器将一个个负责每个动作的模块间隔开来，然后将时钟的周期设置为每个模块刚好可以写入数据的时间。这样一个时钟周期内，每个模块都执行一次完整的动作。在单个时钟周期内，每个模块在服务不同的指令，而不是所有模块服务同一个指令（如果这样则每个模块只在高电平持续时间的一小块时间内工作）。在单位时间内，整个逻辑电路服务的指令总数大大增加，也就是吞吐量得到了增加。

因为增加了电路的复杂性，对于一条指令而言，从头走到尾所需的时间变长了，但对整个电路而言，吞吐量增加了。这便是流水线机制的意义。

流水线听起来很完美，但也存在一些缺陷。比如很难将各个模块的延迟变为一致的，整个电路的速度将受限于最慢的模块。时钟周期必须大于最慢模块的整体计算时间，这就给其它模块带来了延迟。另外，流水线的层级也并非越深越好。随着流水线层架的加深，寄存器的增多将导致整体电路延迟的增加，当层级到达一定深度时，该延迟占用总计算时间的比例增大，造成收益的减小。

指令如流水一样进入处理器，而不是一条指令执行完成后下一条指令才进入处理器。虽然将指令的执行拆分成多个小动作会带来许多麻烦，比如流水线冒险，但其带来的吞吐量及缩短时钟周期的收益是值得花费精力来解决这些麻烦的。

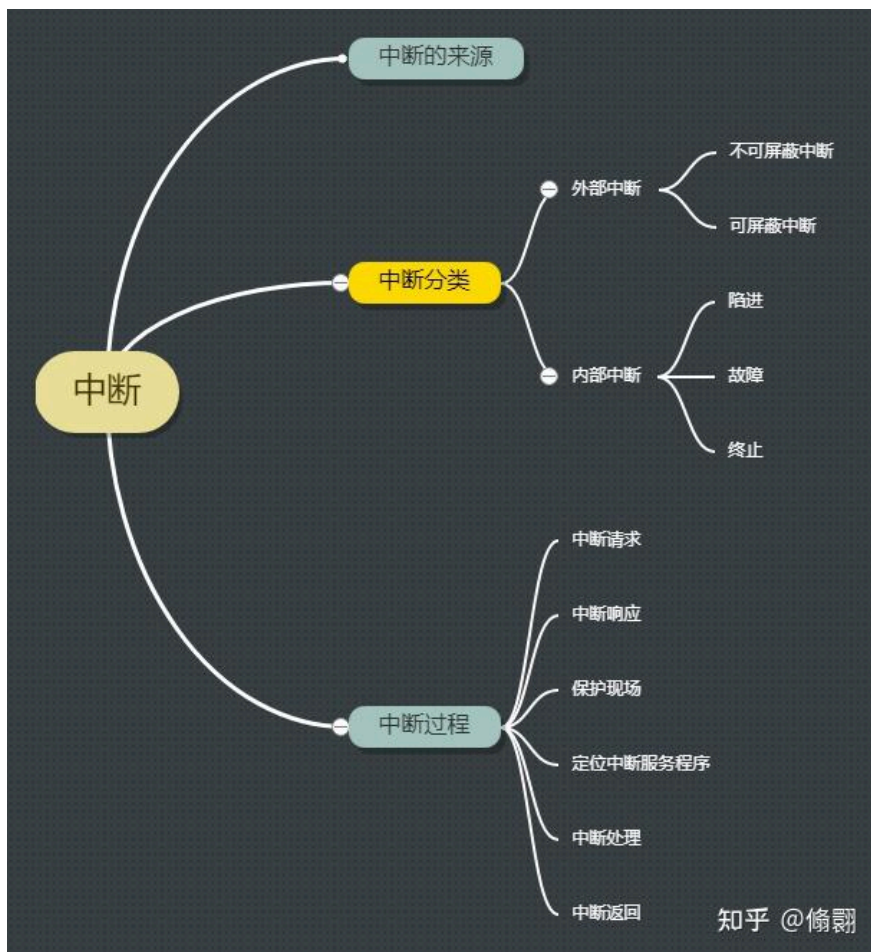
接下来我们讨论周期

机器周期是CPU执行一个基本动作所需要的最小时间。在计算机中，为了便于管理，常把一条指令的执行过程划分为若干个阶段，每一阶段完成一项工作。例如，取指令、存储器读、存储器写等，这每一项工作称为一个基本操作。完成一个基本操作所需要的时间称为机器周期。一般情况下，一个机器周期由若干个S周期（状态周期）组成。通常用内存中读取一个指

指令周期是取出一条指令并执行这条指令的时间。一般由若干个机器周期组成，是从取指令、分析指令到执行完所需的全部时间。指令周期类型有非访内指令的指令周期、取数指令的指令周期、存数指令的指令周期、空操作指令和转移指令的指令周期。需要注意的是，不同指令有不同的指令周期。

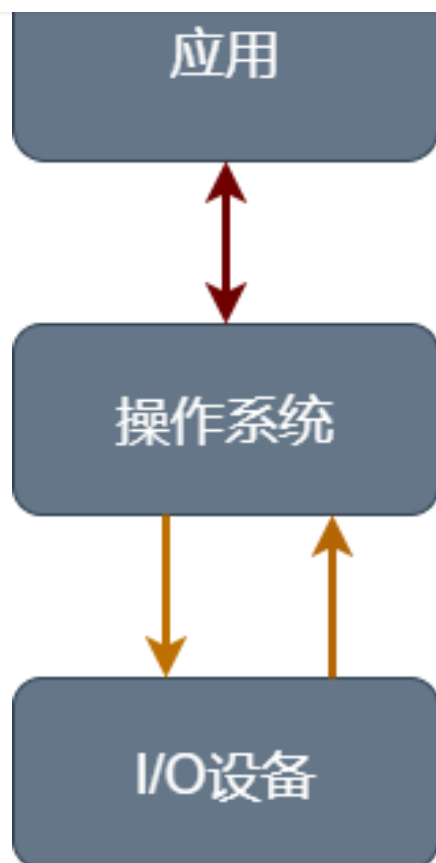
在上述叙述了很多基础概后，中断响应的理解便是建立在这些基础的概念上的。

中断机制



而本文依照如下展开对中断设计的讲解：

在将中断机制处理之前，我们稍微回顾一下CPU的相关处理流程，从而更好的理解中断设计：



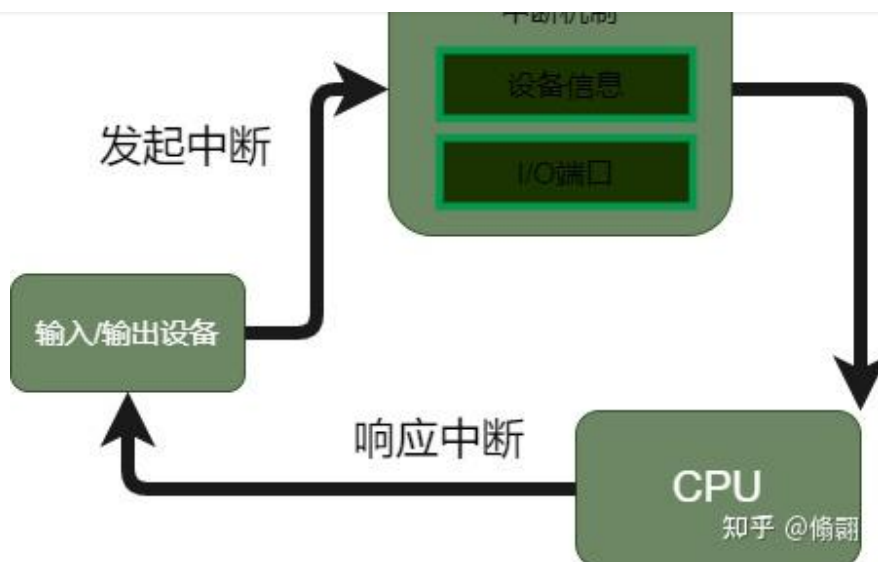
中断的来源

如上图所示，应用通过调用操作系统提供的API与外部进行交互，而API的实现会使用IN或者OUT指令和具体的设备进行通信。但是输入和输出的访问速度，实际上是很慢的，比如从键盘打字到屏幕显示字体，看起来是在一瞬间完成的，但是这若干毫秒的反应速度，CPU可能已经完成了上亿此计算。

CPU与外存直接不会进行直接交互。其主要原因就在于运算和传输数据速度的不同，怕被拖累。面对比外存还慢的外设，和人们更加缓慢的操作，这就需要一种机制来进行交互，称之为中断请求（IRQ，Interrupted Request）。这是一种用来暂停当前程序并跳转到其他程序的必要机制。

那么这个机制同输入/输出设备与CPU的通信有什么关系呢？其关联就在于程序在运行时CPU会不断的执行，而不去理会输入/输出设备中的数据，而I/O设备中断数据需要一致请求CPU的处理。

如下图所示，实施中断请求的是输入/输出设备的I/O控制器，而负责实施中断处理程序的是CPU。



输入/输出设备会不断的发起中断，而不同设备发出的中断请求又不太一样，但是请求需要包括：设备信息和I/O端口。CPU收到中断请求后，会在适当的时间响应中断，发现键盘设备缓冲器有输入的数据，那么将数据拷贝进内存，接着程序继续运作。显示器作为输出设备也会发起中断，当处理对应的中断时，发现内存中有了新数据，就会将新数据输出到显示器进行显示，这样键盘输入的字符就能显示在显示器上了。输入和输出是异步配合的，只不过中断和处理中断很快。

中断是针对程序的，运行中的程序需要暂停，让渡出CPU，而后又切换回来，这个代价其实挺高的。中断需要将当前CPU寄存器（以及栈）内容进行备份，然后将控制权移交给处理输入/输出设备的程序，这个程序一般是操作系统提供用来实现中断处理的，该程序执行完成后，会还原寄存器内容，使得原有程序得以正常运行。

而程序的实现通过不同的寄存器和计数器来实现。在这边不做详细的展开。

总之了解了上述中断的过程，中断简单来说就是让CPU停下当前的工作任务，转手去处理其他事务，完成后在转回来处理这个事务。而一般的中断所说的都是在硬件层的中断。

中断分类

接下来介绍中断的分类，在此我们可以熟悉中断，明白中断类型：

外部中断（硬件层）

一般外部中断，被简称为中断，所以一般说的中断主要是外部中断，通常是异步中断。在外部中断这个类型中，又分为可屏蔽中断和不可屏蔽中断

- 可屏蔽中断：通过INTR线向CPU请求中断，所用I/O设备产生的中断请求均可引起屏蔽中断，主要来自外部设备如硬盘、键盘等。上述中断的例子，也是属于可屏蔽中断。
- 不可屏蔽中断：通过NMI线向CPU请求中断，硬件设备引起的故障。类如电源掉电、硬件电路故障等，这里的不可屏蔽不是指的不能屏蔽，而是不建议屏蔽，因为这类中断带来的影响对于计算机和CPU来说影响很大。

注：INTR和NMI都是CPU的引脚，在这不展开讨论。

内部中断（软中断、异常）

在CPU内部出现的中断称之为内部中断，又被称之为异常，即在CPU执行特定的指令时出现的非法情况。同时异常也被称之为同步中断。因此只有在一条指令执行后才会发出中断，不可能在指令执行期间发生异常。内部中断的分类如下：

回陷阱指令的下一条指令，如系统调度、程序调试等功能。

这里需要注意的是，在平时的编程中可能不常见陷阱，但是并不代表其不存在，应为现在编程所用的高级语言对底层进行了太多次的抽象封装，看不到底层的实现。例如printf函数，最底层的实现中会有一条int 0x80指令，这就是一条陷阱指令，使用0x80号中断进行系统调用。

- 故障：而故障是指引起故障的指令被执行，但是还没有执行结束是，CPU检测到这一类的意外事件。出错的时候交由故障处理程序处理，如果能够修正处理这个命令，就将控制返回到引起故障的指令，即CPU重新执行这个命令；如果不能就处理报错。

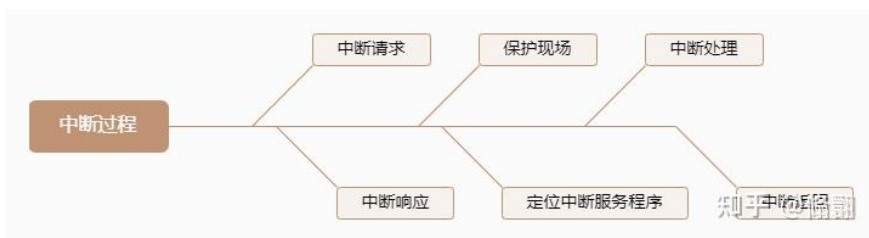
常见的故障为缺页，当CPU引用的虚拟地址对应的物理页不存在时就会发生故障。缺页异常是能够修正的，有着专门的缺页处理程序，它会将缺失的物理页从磁盘中重新调进主存。而后再次执行引起故障的指令时便能够顺利执行了。

- 终止：执行指令的过程中发生了致命错误，不可修复，程序无法继续运行，只能终止，通常会是一些硬件的错误。终止处理程序不会将控制返回给原程序，而是直接终止原程序。

注：CPU对中断的响应并不是靠中断引脚的电平发生了改变，CPU便立即放下手头的事情去做中断源分配的任务，其中还包含着识别中断源、找到中断程序、保存当前任务的各个寄存器状态、进入中断处理程序后的返回等一系列复杂的操作。对于外部中断，CPU在执行当前指令的最后一个时钟周期去查询INTR引脚，若查询到中断请求信号有效，同时在系统开中断（即IF=1）的情况下，CPU向发出的中断请求的外设回送一个低电平有效的中断应答信号，作为对中断请求INTR的应答，系统自动进入中断响应周期（即中断响应指令的指令周期）。

认真阅读了上一章节，那么我们可以理解检查中断引脚对CPU来说也是由一个特定的组合电路模块完成。该模块属于流水线的一部分，处在整个电路的末端，每个时钟周期都在服务着不同的指令。

而检测到中断信号后，上述的识别中断源、找到中断程序、保存当前任务的各个寄存器状态、进入中断处理程序后的返回等操作是由中断隐指令*来完成的。



中断过程

在将中断过程的时候，我们需要了解中断隐指令：

中断隐指令引导CPU在响应中断信号时随机做出的一系列动作，这些动作是在检测到中断信号后便随即发生的，因而不能由软件来完成，而是由硬件来处理。中断隐指令并不是指令系统中的一条真正的指令，它没有操作码，所以中断隐指令是一种不允许、也不可能为用户使用的特殊指令。其所完成的操作主要有：保护现场、定位中断服务程序（暂不允许中断）、中断处理和分发。

中断过程在一般的CPU中是交由特殊的的中断芯片处理的，中断不同，要求和性能也不同，大家可以参考说明书进行学习。

下面介绍一下一般芯片的中断流程：

1. 中断请求

- 当外设发出中断信号后，信号被送入中断芯片
- 检测记否需要处理此中断
- 如果需要处理则想CPU发送INTR信号

1. 中断响应

- CPU收到INTR信号后便知道有新的中断了，在执行完当前指令后，向中断芯片发送一个中断回复信号。

1. 保护现场

- 为了保证在中断服务程序执行完毕能正确返回原来的程序，必须将原来程序的断点（即程序计数器(PC)的内容）保存起来。断点可以压入堆栈，也可以存入主存的特定单元中。

1. 定位中断服务程序

- 暂不允许中断即关中断。在中断服务程序中，为了保护中断现场（即CPU主要寄存器的内容）期间不被新的中断所打断，必须要关中断，从而保证被中断的程序在中断服务程序执行完毕之后能接着正确地执行下去。并不是所有的计算机都在中断隐指令中由硬件自动地关中断，也有些计算机的这一操作是由软件（中断服务程序）来实现的。但是大部分计算机还是靠硬件来进行相关动作，因为硬件具有更好的可靠性和实时性。
- 引出中断服务程序的实质就是取出中断服务程序的入口地址送程序计数器（PC）。对于向量中断和非向量中断，引出中断服务程序的方法是不相同的。

1. 中断处理与返回

- 硬件中断处理。在Windows所支持的硬件平台上，外部I/O中断进入到中断控制器的一根线上。该控制器接着在某一根线上中断处理器。处理器一旦被中断，就会询问控制器以获得此中断请求（IRQ）。中断控制器将该IRQ转译成一个中断号，利用该编号作为索引，在一个称为**中断分发表**（IDT）的结构中找到一个IDT项，并且将控制权传递给恰当的中断分发例程。每个处理器都有单独的IDT，所以，如果合适，不同的处理器可以运行不同的ISR。

中断是操作系统重要的机制，没有中断，操作系统效率将大大降低。最后在这里强调一下：

- 不可以由软件去保护断点，因为当中断以后，cpu的PC值已经被改变了 软件访问不到原来的PC值；但是可以由软件去保护现场，而且好像很多cpu确实也这么做的；恢复断点当然是软件去做的，硬件又不知道你什么时候想返回
- 软件的是顺序执行的，至少是有规律的；但是中断本身就是一个无法预料的事情，程序无法预料何时发生中断，何时执行中断处理函数，所以只能在某个地址写上一段中断处理代码，等中断到来时由硬件将程序强行跳转到你的中断处理函数去处理中断。

参考文章:

[1]博客园 牛有肉 《CPU中断的工作原理》cnblogs.com/niuyourou/p/...

[2]知乎 程序员小灰 《一文讲透计算机的中断》zhuanlan.zhihu.com/p/36...

编辑于 2023-10-19 17:58 · 四川

CPU 设计 中央处理器 (CPU) 中断

▲ 赞同 273 ▼ ● 4 条评论 ↗ 分享 ♥ 喜欢 ★ 收藏 📄 申请转载 ...

写下你的评论...

4 条评论

默认 最新



小小心

下次从电子的角度讲讲😊

2023-03-08 · 上海

● 回复 ♥ 5



回旋的电子

哈哈😄

2023-08-10 · 广东

● 回复 ♥ 喜欢



258CM板凳

太适合我这个新手阅读了，点赞

 **大闹地儿**

好文！适合电路基础不咋地的程序员阅读！

2023-08-22 · 广东

回复 1

推荐阅读

CPU-异常和中断机制

现代计算机中都配有完善的异常和中断处理系统，CPU的数据通路中有相应的异常检测和响应逻辑，外设接口中有相应的中断请求和控制逻辑，操作系统中有相应的中断服务程序。这些中断硬件线路...

Bourne

关于CPU外部中断、边沿or电平、共享中断的总结

在使用CPU的中断连接外设时，CPU端会有两种设置：一种是边沿触发，一种是电平触发。cpu检测电平还是边沿，其本质都是同步逻辑。CPU的中断控制器用内部时钟采外部中断信号，边沿触发是两...

shangyx


C
笔记
张彦飞